RoadWatch

Automated road-damage detection for smarter street maintenance

COURSE

ITAI 1378 — Computer Vision

TEAM

Krhys Killingsworth
Nnaemeka Ofoegbu

TIER

Tier 2 — Fine-tune a pretrained detector

Midterm Project Proposal · Houston Community College

THE PROBLEM

Road inspection is manual, slow, and expensive

$3T+

Deferred U.S. road & bridge maintenance backlog — small defects grow into costly failures.

Manual

Cities still survey pavement by eye or slow specialized vans, so damage is logged late.

Everyone

Drivers, cyclists, transit agencies & public works all bear the cost of bad roads.

Why it matters

Repair cost multiplies with delay. Sealing a crack costs a few dollars per foot; a full pothole repair or resurface costs far more — and unaddressed damage causes crashes, blown tires, and injury claims. In flood-prone Houston, water intrusion accelerates cracking into potholes fast, so early, frequent detection has real safety and budget payoff.

THE SOLUTION

Point a camera at the road — get damage flagged automatically

One sentence: A phone or dashcam captures road images, a fine-tuned YOLO11 detector locates and labels each defect, and the system outputs annotated images plus a damage-count report.

Detects

Draws a bounding box around every crack and pothole in an image or video frame.

Classifies

Labels each defect by type — longitudinal, transverse, alligator crack, or pothole.

Counts & scores

Aggregates detections into per-image counts and a simple severity summary.

Reports

Exports annotated images and a CSV, ready to map or hand to public works.

TECHNICAL APPROACH

Fine-tuned object detection with Ultralytics YOLO11

TECHNIQUE

Object detection

Damage must be located, not just classified — bounding boxes let us count and map defects per image.

MODEL

YOLO11s (Ultralytics)

Proven, fast, and well-documented; the 's' variant fine-tunes on a free Colab/Kaggle GPU. YOLO11m is the upgrade path if accuracy lags.

FRAMEWORK

PyTorch + Ultralytics

One-line train/val/export API, native augmentation, and easy ONNX export for a fast demo.

Why transfer learning? YOLO11 ships with COCO-pretrained weights, so the backbone already understands edges, textures, and shapes. Fine-tuning on road imagery converges in hours on one GPU — realistic for a 6-week midterm.

DATA PLAN

RDD2022 — a public, multi-country road-damage dataset

Source

RDD2022 (CRDDC'2022) — free on Figshare; a YOLO-format mirror is on Roboflow & Kaggle.

Size (scoped)

Full set is 47,420 images / 6 countries. I subset to ~8–10k images (U.S. + Japan + Czech) to fit compute.

Labels

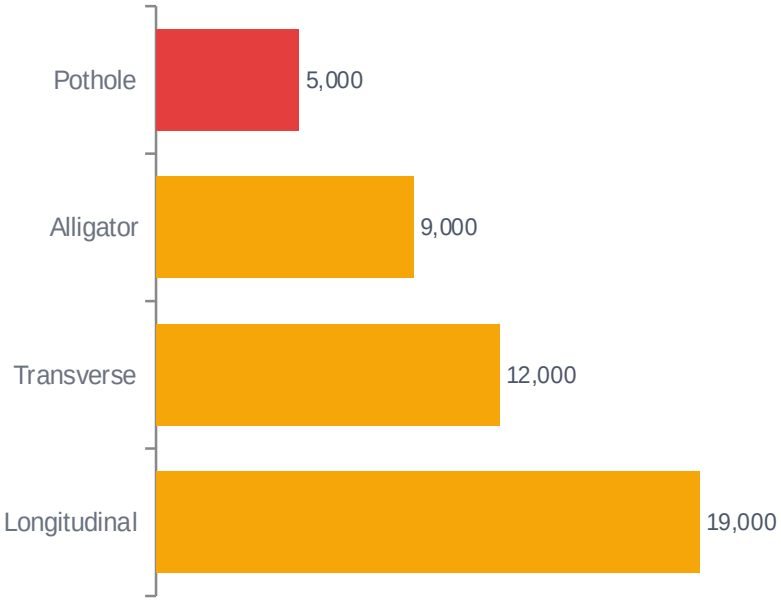
4 defect classes: longitudinal, transverse & alligator cracks (D00/D10/D20) + potholes (D40).

Preparation

Convert VOC XML → YOLO txt, 80/20 train-val split, drop empty images, rebalance rare classes.

Approx. class balance

Cracks dominate; potholes are rarer — a class-imbalance risk to plan for.



SYSTEM DIAGRAM

How an image becomes a damage report

1

Image capture

Phone / dashcam photo or video frame

2

Preprocess

Resize to 640px, normalize

3

YOLO11 inference

Detect + classify each defect

4

Post-process

Filter, count, score severity

5

Output

Annotated image + CSV report

Offline (one time): RDD2022 → clean & convert to YOLO format → fine-tune YOLO11 → export best.pt / ONNX weights. The five steps above run at inference time using those trained weights.

SUCCESS METRICS

How I'll know it works

PRIMARY

mAP@0.5

≥ 0.55

Mean average precision across all 4 classes — the standard detection score.

PRIMARY

Pothole recall

≥ 0.60

Catch the rare, high-severity class — missing potholes is the costly error.

SECONDARY

Inference speed

< 50 ms/img

≈20+ FPS on a T4 GPU so the demo runs on video, not just stills.

Realistic target, not 90%. Detection mAP is a stricter measure than classification accuracy — the CRDDC'2022 winner reached ~F1 0.77 across all six countries. Aiming for mAP@0.5 ≥ 0.55 on a scoped subset is honest and achievable; I'll report precision, recall, and per-class AP, not a single inflated number.

WEEK-BY-WEEK PLAN

Six weeks, one milestone each

W1 — Setup & data

Dataset ready

Download RDD2022, convert to YOLO format, EDA

W2 — Baseline train

Model working

Fine-tune YOLO11s, log first mAP

W3 — Improve

Accuracy up

Augmentation, class rebalancing, tune / try YOLO11m

W4 — Demo build

Demo ready

Inference on local road clips, annotated video + CSV

W5 — Finalize

Everything done

Error analysis, README, docs, final eval

W6 — Present

Presentation day

Slides, dry run, deliver in class

CHALLENGES & BACKUP PLANS

What could go wrong — and Plan B

Risk	Prob.	Why it matters	Mitigation (Plan B)
Class imbalance	Med	Potholes are the rare class, hurting recall.	Oversample + heavy augmentation on potholes; class weights; or merge crack types.
Low mAP	Med	Baseline accuracy below target.	Upgrade YOLO11s → 11m, more epochs, mosaic aug, or narrow to fewer classes.
Compute limits	Med	Colab free GPU timeouts on a big set.	Use Kaggle (30 GPU-hrs/wk), subsample data, checkpoint & resume.
Annotation format	Low	RDD ships VOC XML, YOLO needs txt.	Use Roboflow's pre-converted RDD2022 export or a small conversion script.
Domain gap	Low	Global data ≠ Houston roads.	Collect a small local phone-photo test set for qualitative checks.

RESOURCES NEEDED

Everything runs on free tiers

Compute

Google Colab (free T4) — primary. Kaggle 30 GPU-hrs/wk as backup.

Frameworks

PyTorch, Ultralytics YOLO11, Roboflow, OpenCV, pandas.

Data

RDD2022 — free public dataset (Figshare / Roboflow / Kaggle).

Estimated cost

$0. Optional Colab Pro (~$10/mo) only if free GPU is too limiting.

Feasible in 6 weeks · $0 cost · a real Houston-relevant problem